Lecture 3: Losses, gradients, and gradient descent

COMP 521 • Probability, Optimization, and Scientific Discovery • Sections 2.1–2.4

This handout is one part of the full COMP 521 notes. Section, equation, figure and listing numbers match the full notes, and references to anything outside this handout point there. The notation table is in "How to use these notes" (Lecture 1).

2 The language and mathematics of optimization

2.1 Objective, loss, parameters, constraints

An optimization problem chooses parameters to minimize a scalar objective over a feasible set $\mathcal{F}$:

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}\in\mathcal{F}} \mathcal{L}(\boldsymbol{\theta}). \tag{14}$$

$\min_{\boldsymbol{\theta}} \mathcal{L}$ is the smallest *value*; $\arg\min_{\boldsymbol{\theta}} \mathcal{L}$ is the *location* achieving it. For $\mathcal{L}(\theta) = (\theta - 3)^2 + 2$, the minimum is 2 and the argmin is 3. With ties, the argmin is a set; libraries return one element of it, which is where degeneracies hide.

A *decision variable* is something the optimizer adjusts; a *state variable* describes the system and is computed from the decision variables. In a Stellax calibration, an opacity correction a is a decision variable; the profiles $T(r)$, $\rho(r)$ and abundances $X_i(r)$ are state variables computed by the stellar-structure solver. A *loss* is an objective penalizing misfit; a *regularized objective* adds penalties encoding preferences. The objective encodes what we told the optimizer to value, not what nature values.

2.1.1 Two layers: solving the star and fitting the star

Stellar structure defines the state implicitly through a residual equation

$$\mathbf{R}(\mathbf{s}, \boldsymbol{\theta}) = \mathbf{0}, \tag{15}$$

solved by Newton iterations inside Stellax. This is *root-finding*. Fitting observations is an *outer* optimization over $\boldsymbol{\theta}$, each evaluation of which runs the inner solve. Keep the layers separate. Writing $\|\mathbf{R}\|^2$ as one more term in the outer loss turns exact conservation laws into soft, tunable penalties, and a small residual does not guarantee you are on the physical solution branch.

Worked example 2.1 (A constrained calibration, stated in the right layer)**.** Calibrate $\boldsymbol{\theta} = (\alpha_{\mathrm{MLT}}, f_{\mathrm{ov}}, X_0, Y_0, Z_0)$, where the last three are the *initial* hydrogen, helium and metal mass fractions:

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} \mathcal{L}_{\mathrm{data}}(\boldsymbol{\theta}) \quad \text{subject to} \quad f_{\mathrm{ov}} \geq 0, \quad X_0 + Y_0 + Z_0 = 1, \quad X_0, Y_0, Z_0 \geq 0.$$

An *active* constraint holds with equality at the optimum (for example $f_{\mathrm{ov}}^\star = 0$).

Constraints belong in the right layer

Only the *initial* composition is a calibration parameter. The abundances $X_i(r)$ inside the model star are state variables: the identities $\sum_i X_i(r) = 1$ and $X_i(r) \geq 0$ must hold at every mesh point and every timestep, so they are the solver's job, enforced by construction (for example by evolving all species with a conservative reaction network). Imposing them as constraints on the outer fit would mix the two layers.

2.2 From a noise model to a loss

Let the residuals be $\mathbf{r}(\boldsymbol{\theta}) = \mathcal{G}(\boldsymbol{\theta}) - \mathbf{d}$ (some authors use the opposite sign). The loss should come from a model of how the data were generated, not from habit.

Gaussian noise. If $\mathbf{d} \sim \mathcal{N}(\mathcal{G}(\boldsymbol{\theta}), \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma}$ independent of $\boldsymbol{\theta}$, the negative log-likelihood is, up to a constant,

$$\mathcal{L}(\boldsymbol{\theta}) = -\ell(\boldsymbol{\theta}) = \tfrac{1}{2}\mathbf{r}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\mathbf{r}. \tag{16}$$

The inverse covariance down-weights noisy and redundant data. For independent errors $\boldsymbol{\Sigma}$ is diagonal and $\mathcal{L} = \frac{1}{2}\sum_i (r_i/\sigma_i)^2 = \frac{1}{2}\chi^2$. In code it is cleanest to *whiten*: with a Cholesky factor $\boldsymbol{\Sigma} = \mathbf{C}\mathbf{C}^{\mathsf{T}}$, define $\tilde{\mathbf{r}} = \mathbf{C}^{-1}\mathbf{r}$, so $\mathcal{L} = \frac{1}{2}\|\tilde{\mathbf{r}}\|^2$ and all later formulas hold with $\boldsymbol{\Sigma} = \mathbf{I}$.

Parameter-dependent noise

If the covariance depends on parameters (a fitted "jitter" term, or model-dependent theoretical errors), the negative log-likelihood is $\frac{1}{2}\mathbf{r}^{\mathsf{T}}\boldsymbol{\Sigma}(\boldsymbol{\theta})^{-1}\mathbf{r} + \frac{1}{2}\log\det\boldsymbol{\Sigma}(\boldsymbol{\theta}) + \text{const}$. Dropping the log-determinant lets the optimizer reduce χ^2 by inflating the errors without limit.

Counts. For photon or star counts n_i with model expectations $\mu_i(\boldsymbol{\theta})$, the Poisson likelihood gives $\mathcal{L} = \sum_i [\mu_i - n_i \log \mu_i]$ (plus a constant). Replacing it with χ^2 biases fits wherever counts are small. Censored data, upper limits, and contaminated samples each need their own likelihood.

Worked example 2.2 (A complete TOY calibration)**.** Let a normalized observable obey the TOY local relation $y(\theta) = 1 + 2\theta$, and observe $d = 3$ with $\sigma = 0.5$. Then $\mathcal{L}(\theta) = \frac{1}{2}\big((1+2\theta-3)/0.5\big)^2 = 8(\theta-1)^2$, with minimizer $\theta^\star = 1$ and minimum 0. We use this loss to study algorithms before substituting an expensive Stellax calculation.

2.3 Gradient, Jacobian, Hessian

For a scalar $\mathcal{L}(\boldsymbol{\theta})$ with p parameters, the *gradient* $\nabla\mathcal{L} \in \mathbb{R}^p$ collects $\partial\mathcal{L}/\partial\theta_j$. It points in the direction of steepest increase *for the Euclidean metric in the coordinates you chose*; rescale a parameter and the steepest direction changes. For a vector function $\mathcal{G} : \mathbb{R}^p \to \mathbb{R}^m$, the *Jacobian* is the $m \times p$ matrix $J_{ij} = \partial\mathcal{G}_i/\partial\theta_j$. For a scalar function, the *Hessian* $H_{ij} = \partial^2\mathcal{L}/\partial\theta_i\partial\theta_j$ describes curvature. A zero gradient with positive-definite Hessian guarantees a strict local minimum; a zero gradient alone could be a minimum, maximum, or saddle.

Dimensions are a free check. If Stellax maps 5 parameters to 20 observables, $\mathbf{J}$ is 20×5, but the gradient of a scalar loss built from those observables has length 5.

2.3.1 The least-squares chain rule

For $\mathcal{L} = \frac{1}{2}\mathbf{r}^\top \boldsymbol{\Sigma}^{-1}\mathbf{r}$ with fixed $\boldsymbol{\Sigma}$ and $\mathbf{J} = \partial \mathbf{r}/\partial \boldsymbol{\theta}$,

$$\nabla \mathcal{L} = \mathbf{J}^\top \boldsymbol{\Sigma}^{-1}\mathbf{r}, \qquad \mathbf{H} = \underbrace{\mathbf{J}^\top \boldsymbol{\Sigma}^{-1}\mathbf{J}}_{\mathbf{H}_{\mathrm{GN}}} + \sum_i (\boldsymbol{\Sigma}^{-1}\mathbf{r})_i \nabla^2 r_i. \tag{17}$$

The *Gauss–Newton* matrix $\mathbf{H}_{\mathrm{GN}}$ drops the second term, which is small when residuals are small or the model is nearly linear. It is always positive semi-definite, needs only first derivatives, and, as Section 4.2 shows, is exactly the Fisher information. Remember the matrix $\mathbf{J}^\top \boldsymbol{\Sigma}^{-1}\mathbf{J}$: it appears three more times.

2.4 Gradient descent and the step size

Gradient descent steps against the gradient:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}_k), \tag{18}$$

with step size (learning rate) η_k. For $\mathcal{L} = 8(\theta - 1)^2$, $\mathcal{L}' = 16(\theta - 1)$, so $\theta_{k+1} - 1 = (1 - 16\eta)(\theta_k - 1)$. The error shrinks iff $|1 - 16\eta| < 1$, i.e. $0 < \eta < 1/8$; $\eta = 1/16$ lands on the minimum in one step; $\eta = 1/8$ oscillates forever between two points.

The general lesson: conditioning sets the speed. Near a minimum every smooth loss looks like a quadratic $\frac{1}{2}\boldsymbol{e}^\top \mathbf{H}\boldsymbol{e}$ in the error $\boldsymbol{e} = \boldsymbol{\theta} - \boldsymbol{\theta}^\star$. Then $\boldsymbol{e}_{k+1} = (\mathbf{I} - \eta\mathbf{H})\boldsymbol{e}_k$, and along each eigenvector of $\mathbf{H}$ (eigenvalue λ_i) the error is multiplied by $1 - \eta\lambda_i$ per step. Hence:

- stability requires $\eta < 2/\lambda_{\max}$: the stiffest direction caps the step;
- the best fixed step, $\eta = 2/(\lambda_{\min} + \lambda_{\max})$, contracts the error by $(\kappa - 1)/(\kappa + 1)$ per iteration, where $\kappa = \lambda_{\max}/\lambda_{\min}$ is the condition number;
- reaching a tolerance therefore takes roughly $\frac{\kappa}{2}\ln(1/\mathrm{tol})$ iterations.

ex03_conditioning.py -- iterations needed grow linearly with the condition number

```python
"""Gradient descent on an ill-conditioned quadratic, and what rescaling does.

L(theta) = 1/2 theta^T H theta with H = diag(1, kappa).  For fixed step eta,
GD converges iff eta < 2/lambda_max; the best fixed step gives the error
contraction factor (kappa - 1)/(kappa + 1) per iteration.
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)


def gd_iterations(kappa, tol=1e-6, max_iter=100_000):
    H = jnp.diag(jnp.array([1.0, kappa]))
    loss = lambda th: 0.5 * th @ H @ th
    grad = jax.jit(jax.grad(loss))
    eta = 2.0 / (1.0 + kappa)  # optimal fixed step for this quadratic
    th = jnp.array([1.0, 1.0])
    for k in range(max_iter):
```

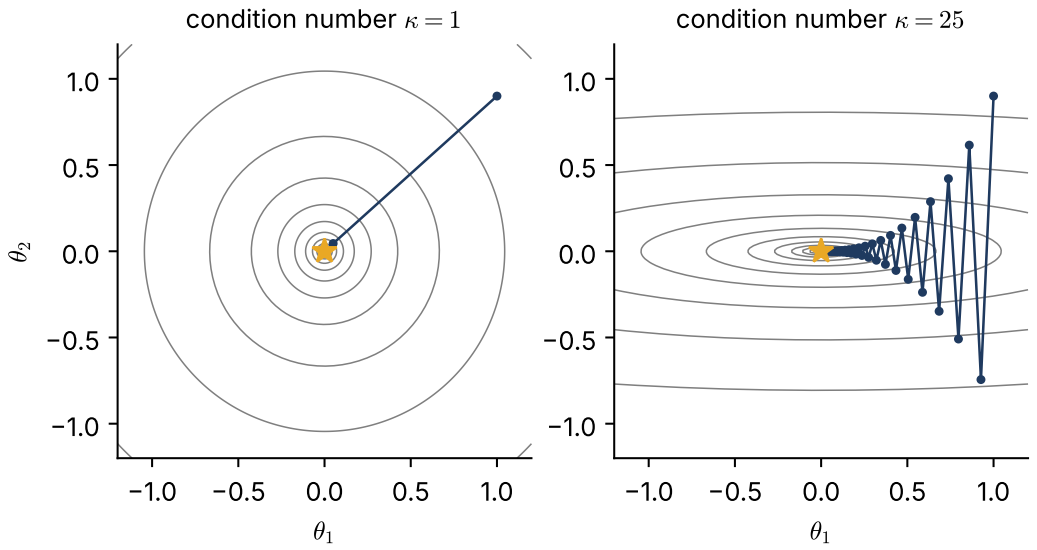


Figure 3: Gradient descent with a near-optimal fixed step. Left: a round bowl ($\kappa = 1$). Right: an elongated valley ($\kappa = 25$); the step is limited by the steep direction, so progress along the shallow direction is slow and the iterates zig-zag.

```
        if jnp.linalg.norm(th) < tol:
            return k
        th = th - eta * grad(th)
    return max_iter


for kappa in [1.0, 10.0, 100.0, 1000.0]:
    print(f"kappa = {kappa:7.0f}: GD needs {gd_iterations(kappa):6d} iterations;"
          f" predicted contraction per step = {(kappa - 1) / (kappa + 1):.4f}")

# Newton's method solves H d = -g and lands on the minimum of a quadratic in one step
H = jnp.diag(jnp.array([1.0, 1000.0]))
th0 = jnp.array([1.0, 1.0])
th1 = th0 + jnp.linalg.solve(H, -(H @ th0))
print("Newton step from (1,1):", th1)

# Rescaling z = sqrt(H) theta turns the kappa=1000 problem into kappa=1:
# in z-coordinates the Hessian is D^-1 H D^-1 with D = sqrt(H)
D_inv = jnp.diag(1.0 / jnp.sqrt(jnp.diag(H)))
print(f"cond(H) = {jnp.linalg.cond(H):.1f} -> after diagonal rescaling:"
      f" {jnp.linalg.cond(D_inv @ H @ D_inv):.1f}")
```

Output (executed for these notes):

```
kappa =       1: GD needs      1 iterations; predicted contraction per step = 0.0000
kappa =      10: GD needs     71 iterations; predicted contraction per step = 0.8182
kappa =     100: GD needs    709 iterations; predicted contraction per step = 0.9802
kappa =    1000: GD needs   7082 iterations; predicted contraction per step = 0.9980
Newton step from (1,1): [0. 0.]
cond(H) = 1000.0 -> after diagonal rescaling: 1.0
```

Momentum methods (heavy ball, Nesterov acceleration) improve the dependence to roughly $\sqrt{\kappa}$. An *iteration* is one update. An *epoch*, in machine learning, is one pass through a finite training set, which may contain many iterations when data are processed in mini-batches.

Exercises for this lecture

Exercises 1, 7, numbered as in Appendix A of the full notes. Worked solutions are in Appendix B; attempt each one before reading its solution.

1. [P] Sets of minimizers. Let $\mathcal{L}(\theta) = \theta^4 - 2\theta^2$. Find $\min_\theta \mathcal{L}$, the set $\arg\min_\theta \mathcal{L}$, and every stationary point with its type. What would a gradient-based library return if started at $\theta_0 = 0$? At $\theta_0 = 10^{-6}$? What does this suggest about reporting a single best fit when a model has a symmetry?

7. [P] Stability on an anisotropic bowl. For $\mathcal{L} = \frac{1}{2}(\theta_1^2 + 50\,\theta_2^2)$: (a) the largest stable fixed step; (b) the best fixed step and its contraction factor; (c) roughly how many iterations reduce the error by 10^6; (d) how many Newton steps; (e) a change of variables that makes gradient descent converge in one step.